

Cloud File Archive (CFA) Specification

Version 1.0 · 2026-09-24 · First published version

Abstract

Cloud File Archive (CFA) is an open, self-describing format for backing up a local file tree to a Google Cloud Storage bucket as a set of POSIX tar archives with companion JSON indexes. This document specifies the on-bucket format and the backup/restore behavior an implementation must follow, so that independent tools can read — and, with care, write — a CFA archive without access to any particular implementation's internal database. It is intended for engineers building or auditing a CFA-conformant backup or restore tool.

Status of this document

This is the first published version of the CFA specification. Conformance language ("MUST", "MUST NOT", "SHOULD", "MAY") follows RFC 2119.

1. Overview

CFA backs up files from a source tree to a Google Cloud Storage (GCS) bucket as POSIX tar archives, each with a companion JSON index. A reader needing only "is this file backed up, and how do I get it back" needs sections 2-7; sections 8-10 cover encryption, compression, and storage-tier selection, each independently optional.

Core requirements:

1. Every backed-up file **MUST** have a SHA-256 computed over its original (pre-compression, pre-encryption) bytes.
2. Every file **MUST** be stored inside a tar archive, never as a bare object.
3. Files smaller than `min_size` **MUST** be clumped together into shared archives; files larger than `max_size` **MUST** be split into parts.
4. Every archive **MUST** have a companion JSON index, written only after the archive's upload is confirmed by size and CRC32C.
5. An archive with no confirmed index **MUST** be treated as incomplete and ignored by restore.

6. Objects already written to the bucket by a conforming implementation MUST NOT be deleted, overwritten, or rewritten by a later run. New data is always written under a fresh archive ID.

2. Settings

Setting	Default	Notes
<code>min_size</code>	2.5 MiB	Files smaller than this are clumped
<code>clump_size</code>	64 MiB	A clump archive is closed once its packed payload would reach this size
<code>max_size</code>	1 GiB	No uploaded archive's payload exceeds <code>max_size</code> - 1 MiB (§3.1)
<code>bucket</code> , <code>prefix</code>	none	For example <code>cfa-backup</code> , <code>server01/</code>
<code>storage_class</code>	bucket default	GCS storage class new objects are written with; see §10
<code>sources</code>	none	Local paths to back up, plus exclude globs
<code>compression</code>	disabled	Optional per-archive gzip; see §9
<code>encryption</code>	disabled	Optional per-file AES-256-GCM; see §8

Constraints an implementation MUST enforce: `clump_size` ≤ `max_size`, and `max_size` MUST leave at least 1 MiB of headroom (§3.1) — implementations MUST reject a `max_size` that does not.

3. Packing

Every file is classified into exactly one archive type by its size:

File size	Archive type	Contents
< <code>min_size</code>	<code>clump</code>	Many files, added in path order until the archive's packed payload would reach <code>clump_size</code>

File size	Archive type	Contents
<code>min_size</code> to <code>payload_ceiling(max_size)</code>	<code>single</code>	One file
<code>> payload_ceiling(max_size)</code>	<code>part</code>	One slice of one file, per archive

3.1 The payload ceiling

Tar format overhead (a 512-byte header per member, padding to a 512-byte block, end-of-archive padding, and a PAX extended header for long names) means a tar whose payload is exactly `max_size` bytes produces an object larger than `max_size`. To guarantee no archive object ever exceeds `max_size`, every packing decision is made against a payload ceiling:

```
payload_ceiling(max_size) = max_size - 1 MiB
```

This ceiling is the `single / part` boundary, the split chunk size (§3.2), and — together with `clump_size` — the clump seal point:

```
clump_ceiling = min(clump_size, payload_ceiling(max_size))
```

1 MiB comfortably covers measured PAX overhead (worst case observed: ~13 KiB); it is not itself the overhead value, and implementations **MUST NOT** substitute a smaller margin without re-measuring against their own tar writer.

3.2 Splitting

A file larger than `payload_ceiling(max_size)` is split into `k = ceil(size / payload_ceiling(max_size))` parts, each archived separately. Parts are **balanced**, not chunk-sized with a small remainder: each part is `ceil(size / k)` bytes, except that the arithmetic may leave the last part smaller. (Encrypted backups cut parts on ciphertext chunk boundaries instead — see §8.3 — trading balanced sizes for boundaries a restore can stream one part at a time.)

3.3 Tar format and member names

Archives use PAX-format POSIX tar (`tarfile.PAX_FORMAT` or equivalent), not strict ustar: ustar's 100-character member-name limit is routinely exceeded by real media paths. A conforming reader **MUST** support PAX extended headers. No compression is applied at the tar level (§9 compresses file contents before packing, not the tar itself).

- **Plain (unencrypted) archives:** the tar member name is the file's path relative to its source root. A gzip-compressed member's name has a `.gz` suffix appended. A part's member name has `.partNNNN` appended (numbered from `0001`).
- **Encrypted archives (§8):** the tar member name is `<sha256>.file` (`.partNNNN` appended for a part) — paths never appear in the tar itself.

A conforming reader extracts a `clump` or `single` archive with an ordinary tar tool (`tar -xf`). For a `part` archive, concatenate the parts' *decoded* bytes in `part` order (see §8/§9 for what decoding means when encryption or compression is in use); for a plain, uncompressed file this is a plain `cat` of the extracted members.

Leftover small files at the end of a run go into a final, possibly smaller, clump.

4. Object Layout

Each archive is written as a same-directory, same-basename pair — a tar object and a JSON index object differing only in extension:

```
<prefix><archive_id>.tar
<prefix><archive_id>.json
```

`archive_id` is a UUID4, generated fresh for every archive. `<prefix>` is the configured bucket prefix, optionally followed by a per-library subfolder; a conforming implementation may serve several libraries under one bucket, each in its own subfolder.

A reader encountering `<prefix>archives/<archive_id>.tar` and `<prefix>index/<archive_id>.json` (an archive/ and index/ split at the folder level) **MUST** also treat that as a valid, complete archive — this was this format's original layout, and existing archives written under it are not rewritten. New archives always use the sidecar form above; the two forms **MAY** coexist indefinitely within one prefix.

5. Index File

```
{
  "v": 1,
  "archive_id": "3f1c2a7e-5d0b-4f8e-9a61-0c2b7d4e9f10",
  "object": "server01/3f1c2a7e-5d0b-4f8e-9a61-0c2b7d4e9f10.tar",
  "type": "clump",
  "created_at": "2026-09-16T12:00:00Z",
  "size": 67108864,
  "encrypted": false,
  "files": {
    "9e1f...a7": {
      "paths": ["photos/2024/img001.jpg"],
      "size": 2048576,
      "mtime": "2026-01-02T03:04:05Z",
      "member": "photos/2024/img001.jpg",
      "offset": 512,
      "compression": "gzip"
    }
  }
}
```

- `v` is the index format version (currently `1`).
- `files` is keyed by the file's SHA-256 (hex), computed over the file's **original** bytes — before any compression or encryption. If two files in the same run share a hash, the content is stored once and `paths` lists both.
- `size` is always the whole file's original size, even for a `part` entry — this is what lets a hash lookup across index files find every part of a split file. It is **not** the byte length of that member; see §7.2 for what a restorer must use instead.
- `offset` is the byte offset of the member's data within the tar. For `clump` and `single` archives this, together with `size`, supports a ranged read of the exact member. It is not meaningful as a read length for `part` archives (§7.2).
- `compression` is present (`"gzip"`) only when that file's stored bytes are gzip-compressed; its absence means stored-as-is. See §9.
- `encrypted` is present and `true` only on an encrypted archive's index (§8), in which case every entry carries `paths_enc` instead of `paths`.
- For `part` entries, each also carries `"part": n` and `"parts": k` (1-indexed, `k` total). The keyed hash is that of the whole file, so a hash lookup across an archive set finds all of a split file's parts.

6. Backup Run

1. Scan the sources. Skip any file whose path, size, and mtime match the last successful backup for it at this archive.
2. Hash each remaining file (streaming SHA-256, over its original bytes). If the file changes during reading (size or mtime differs afterward), skip it for this run rather than backing up a torn read.
3. **Content reuse:** if a file's SHA-256 already has a complete backup at this archive (the same file renamed, moved, copied, or merely touched) an implementation MAY record it against the existing stored bytes instead of re-uploading — subject to that existing copy being at the same encryption state and (if encrypted) key version. This changes which files reach step 4, not the index or archive formats themselves.
4. Apply compression (§9) and encryption (§8), if enabled, to each remaining file's bytes, in that order.
5. Pack the resulting bytes into archives (§3). Write each archive to a local staging area.
6. Upload each archive; confirm the object's size and CRC32C against what was staged before considering the upload done. A failed upload SHOULD be retried with backoff (a reference implementation retries up to 5 times); an implementation that exhausts its retries MUST abort the run without writing that archive's index — do not silently drop the archive and continue.
7. Only after an upload is confirmed, write that archive's index file (§5), in the storage class configured for the archive (§10).
8. Record the archive and its files in the backup ledger.

Because index files are written strictly after a confirmed upload, and never before, a reader can trust: **archive object present + index object present + index parses ⇒ that archive is complete**. An archive with a missing or unparsable index MUST be treated as incomplete and ignored by restore, regardless of whether the `.tar` object exists.

Only one backup run may execute concurrently against the same bucket prefix; implementations SHOULD serialize with a lock scoped to the whole run, not per-archive, since concurrent runs would race on which archive IDs exist without providing any benefit (archive IDs are already unique per run).

7. Restore

7.1 Locating a file's parts

Look up the file by hash (or path) in the backup ledger to get the ordered list of archives and offsets that reconstruct it. A from-ledger restore is the only mechanism this version of the specification defines; rebuilding a file's location purely by scanning bucket index files (with no ledger available) is out of scope for v1.0 — see §11.

7.2 Reading each part

- **clump or single member:** a ranged read of `size` bytes at `offset` within the archive's `.tar` object yields exactly the member's stored bytes (tar stores each member's data contiguously starting at its header's data offset) — no tar parsing is needed.
- **part member:** the index's `size` field is the whole file's size, not this part's byte length, and is not a valid read length here. Download the whole `part` archive object and use ordinary tar member parsing (the archive holds exactly one tar member) to get that part's exact bytes.

7.3 Reassembly and verification

Concatenate a file's parts, in `part` order, into one stream of stored bytes. If the file was encrypted (§8), decrypt that stream. If it was compressed (§9), gunzip the (now-plaintext) result. Compute the SHA-256 of the final bytes and compare it against the file's recorded hash before treating the restore as successful — a mismatch (including a decrypt failure) MUST be reported as a restore failure, not written into place.

8. Encryption (optional)

When enabled, every file backed up from then on is client-side encrypted before packing. Encryption is a whole-pipeline setting, not chosen per-file: an archive's index carries `"encrypted": true` when it applies.

8.1 Key derivation

- A master key is derived from an operator-supplied passphrase via PBKDF2-HMAC-SHA256 (200,000 iterations in the reference implementation), using a salt fixed at the time the passphrase is first set.

- Each file's encryption key is derived deterministically: `file_key = HMAC-SHA256(master_key, "cfa-file-key:" + sha256_hex)`. Nothing per-file needs to be stored to decrypt later — the passphrase (or the key version that encrypted it; implementations MAY support rotating the passphrase and recording which version encrypted each archive) and the file's already-catalogued hash are sufficient.

Because the key is derived from the file's own hash, this is deliberately not semantically secure in the strict sense (two files with identical content always derive the same key) — this is an accepted tradeoff for this format, not an oversight; each encryption still uses a fresh random nonce prefix (below), so GCM's nonce-uniqueness requirement is not violated by it.

8.2 Blob format

A file is encrypted into exactly one blob before any packing:

```
header (20 bytes) | chunk 0 | chunk 1 | ...
```

```
header = "MBE1" (4 bytes) | nonce_prefix (4 bytes) | chunk_size: u32 BE |
plain_size: u64 BE
chunk i = AES-256-GCM(file_key, nonce_prefix || i as u64 BE,
plaintext_chunk_i, aad = header)
```

- `nonce_prefix` is 4 random bytes, freshly generated for every encryption of that file (so re-backing up a changed file gets a new prefix and no nonce is ever reused for a given key).
- Every chunk except the last is exactly `chunk_size` plaintext bytes, encrypting to `chunk_size + 16` bytes (the GCM tag). The reference chunk size is 64 MiB.
- The header is authenticated as AAD on every chunk, so a tampered `chunk_size` or `plain_size` fails authentication rather than silently mis-decoding.

A conforming decryptor MUST reject a blob whose magic is not `MBE1`, and MUST treat any `InvalidTag` (an authentication failure on any chunk) as a hard failure — never emit partial plaintext from a failed chunk.

8.3 Interaction with packing

Clumping and splitting (§3) act on the **ciphertext** blob, not the original file. Because every chunk but the last is a fixed encrypted size, a `part` archive boundary for an encrypted file is chosen to land exactly on a chunk boundary (as many whole chunks as fit under the

payload ceiling, the first part also carrying the header) rather than at the balanced split point §3.2 uses for plaintext — this lets a restorer decrypt a part at a time without buffering the whole file.

8.4 Index

An encrypted archive's index (§5) replaces every entry's `paths` list with `paths_enc`: the file's real path(s), sealed with that file's own key (hash used as associated data), so a plaintext path never appears at rest. Hashes, sizes, offsets, and mtimes remain plaintext in the index — a private bucket is assumed to be the confidentiality boundary for that metadata; an implementation with a stronger threat model **MUST NOT** rely on this specification for path confidentiality beyond that.

8.5 Restore

Decrypt per §7.3: reassemble the ciphertext stream from its parts, verify and decrypt each chunk in order using the file's derived key, and treat any authentication failure, truncation, or reordering as a restore failure.

9. Compression (optional)

When enabled, a file's bytes **MAY** be gzip-compressed (RFC 1952) before encryption (§8) and packing (§3) — compression always precedes encryption, never the reverse, since ciphertext does not compress. An implementation **SHOULD** skip compression (store the file as-is) for:

- File extensions it knows are already compressed (video, image, and audio formats already using lossy or entropy coding; zip-family archives).
- Any file whose compressed size does not save at least a threshold fraction of the original (the reference implementation uses 5%, checked against a sample rather than compressing the whole file when a quick estimate already fails the threshold).

Output is a standard gzip stream decodable by any conforming gzip implementation, with no embedded filename or timestamp (i.e. deterministic output for identical input). The file's SHA-256 (§5, §6) is always over the **original**, uncompressed bytes — compression state has no bearing on identity, dedup, or key derivation.

An index entry for a compressed member carries `"compression": "gzip"`; its absence means the stored bytes are the original file as-is. Restore gunzips **after** decrypting (§7.3, §8.5) — compression is the innermost transform applied on backup and so the outermost

removed on restore.

10. Storage Class

Each archive (a bucket + prefix, or a per-library subfolder within one) MAY be configured with a GCS storage class — `STANDARD`, `NEARLINE`, `COLDLINE`, or `ARCHIVE` — applied to every object (`.tar` and `.json`) written to it from that point on. Leaving it unset means new objects take the bucket's own default class.

- The storage class is a property of the archive's write path going forward, not of the bucket or any individual object retroactively: it **MUST NOT** be changed after archives already exist under that configuration, and implementations **MUST NOT** rewrite the class of already-written objects when it is changed for future writes. (A reference implementation enforces this by making the choice permanent at the time an archive is created.)
- Colder classes (`NEARLINE` / `COLDLINE` / `ARCHIVE`) carry provider-side minimum storage durations and retrieval costs. A restore or a deep verify (reading object bytes back) against such an archive incurs those costs; this specification does not change or hide that.
- A listing of an archive's objects (for inventory or reconciliation purposes) **SHOULD** report each object's storage class, since one archive's objects MAY legitimately span multiple classes if the setting was introduced after some objects already existed at the bucket default.

11. Out of Scope

The following are explicitly not defined by this version of the specification. An implementation MAY provide them as an extension, but **MUST NOT** claim CFA v1.0 conformance for behavior that contradicts §1's core requirements while doing so.

- Version history or retention policy for old file content.
- Deletion or garbage collection of archives no longer referenced by any current file (in particular, note §6.3: stored bytes may be shared by more than one logical file, so any future deletion mechanism must reference-count before removing an archive).
- Compaction or rewriting of existing archives.
- Restoring purely from bucket contents with no backup ledger available (a from-index-only, database-less recovery path). This is deliberately deferred rather than assumed solvable by a small extension: it needs a bounded-cost way to find a hash across an

unbounded number of index files, which this version does not specify.

- Windows-specific file metadata (ACLs, alternate data streams, etc.).
- Storage providers other than Google Cloud Storage.
- Key rotation mechanics beyond "an implementation MAY record which key version encrypted a given archive" (§8.1) — the rotation workflow itself (how a new passphrase is introduced, whether old archives are re-encrypted) is not specified.

12. Conformance Tests

An implementation claiming CFA v1.0 conformance should be able to demonstrate:

1. Round trip: files of 0 bytes, just under and just over `min_size`, and over `max_size`, restore byte-identical to their originals — with encryption and compression each independently on and off.
2. No uploaded archive object exceeds `max_size`.
3. `clump` and `single` archives extract with a standard `tar -xf`; `part` archives extract correctly with a standard tar tool with parts reassembled per §7.2-7.3.
4. Every index entry's hash, offset, and size are consistent with its archive's actual bytes.
5. An interrupted run leaves no index file for any archive whose upload was not confirmed.
6. An unchanged file (same path, size, and mtime as its last backup) is not re-uploaded on a subsequent run.
7. A file whose content already exists at the archive under a different path is not re-uploaded (§6.3), and both paths are still resolvable via the ledger.
8. A tampered or truncated encrypted blob fails restore with an explicit error, never with silently wrong plaintext.
9. A file compressed on backup restores to its original, uncompressed bytes, with its recorded hash matching the original file, not the compressed stream.